MLT- Lab 1

Title:

Decision tree and random forest implementation and comparison

Dataset Descriptiom:

- ex: Iris
  - sepal_length,sepal_width,petal_length,petal_width
  - 3 classes
- ex: Breast Cancer
  - radius_mean,texture_mean,perimeter_mean,area_mean,smoothness_mean,compactness_mean,concavity_mean,concave_points_mean,symmetry_mean,fractal_dimension_mean,radius_se,texture_se,perimeter_se,area_se,smoothness_se,compactness_se,concavity_se,concave_points_se,symmetry_se,fractal_dimension_se,radius_worst,texture_worst,perimeter_worst,area_worst,smoothness_worst,compactness_worst,concavity_worst,concave_points_worst,symmetry_worst,fractal_dimension_worst
  - 2 classes
- ex: Wine
  - alcohol,malic_acid,ash,alcalinity_of_ash,magnesium,total_phenols,flavanoids,nonflavanoid_phenols,proanthocyanins,color_intensity,hue,od280/od315_of_diluted_wines,proline
  - 3 classes

Pseudo code:

1. Decision Tree:
   DecisionTreeBuild(X, y, depth, max_depth, min_samples_split)
   if (depth >= max_depth) or (len(y) < min_samples_split) or (all y are same class)
   return LeafNode(most_common_class in y)
   best_feature, best_threshold = FindBestSplit(X, y)
   left_mask = X[best_feature] <= best_threshold
   right_mask = ~left_mask
   left_tree = DecisionTreeBuild(X[left_mask], y[left_mask], depth+1, max_depth, min_samples_split)
   right_tree = DecisionTreeBuild(X[right_mask], y[right_mask], depth+1, max_depth, min_samples_split)
   return TreeNode(best_feature, best_threshold, left_tree, right_tree)
2. Random Forest:
   RandomForestBuild(n_estimators, X, y, max_depth, min_samples_split, max_features)

```
forest = empty list
for i from 1 to n_estimators
bootstrap_X, bootstrap_y = BootstrapSample(X, y)
tree = DecisionTreeBuild(bootstrap_X, bootstrap_y, 0, max_depth, min_samples_split)
add tree to forest
return forest
```

Result analysis:

Settings	Dataset 1 (Iris - 3 classes, 4 attributes)	Dataset 2 (Breast Cancer - 2 classes, 30 attributes)	Dataset 3 (Wine - 3 classes, 13 attributes)
Default (Gini, max_depth=3, min_samples_split=2)	Scratch DT: 0.978 Sklearn DT: 0.978 Scratch RF: 0.956 Sklearn RF: 0.978 **Effects**: Low depth prevents overfitting, very fast training, RF slightly lower due to small dataset but stable.	Scratch DT: 0.924 Sklearn DT: 0.930 Scratch RF: 0.947 Sklearn RF: 0.959 **Effects**: Good balance, RF reduces variance significantly on higher-dimensional data.	Scratch DT: 0.907 Sklearn DT: 0.926 Scratch RF: 0.944 Sklearn RF: 0.963 **Effects**: RF improves multi-class stability, moderate training time.
Entropy Criterion (instead of Gini)	Scratch DT: 0.978 Sklearn DT: 0.978 Scratch RF: 0.967 Sklearn RF: 0.978 **Effects**: Slightly better split quality on multi-class, minor increase in training time (~20%).	Scratch DT: 0.918 Sklearn DT: 0.924 Scratch RF: 0.953 Sklearn RF: 0.965 **Effects**: Small accuracy boost in RF, deeper trees possible, useful for imbalanced features.	Scratch DT: 0.889 Sklearn DT: 0.907 Scratch RF: 0.944 Sklearn RF: 0.963 **Effects**: Comparable to Gini, no major difference on this dataset.
Max Depth = None (Full tree growth)	Scratch DT: 0.956 Sklearn DT: 0.978 Scratch RF: 0.978 Sklearn RF: 0.978 **Effects**: Single DT overfits (train acc 1.0), RF mitigates overfitting effectively. Longer training time.	Scratch DT: 0.906 Sklearn DT: 0.912 Scratch RF: 0.959 Sklearn RF: 0.971 **Effects**: Severe overfitting in DT on high-dim data, RF remains robust.	Scratch DT: 0.870 Sklearn DT: 0.889 Scratch RF: 0.963 Sklearn RF: 0.981 **Effects**: Overfitting visible in DT, RF handles it well with bagging.

Min Samples Split = 5	Scratch DT: 0.978 Sklearn DT: 0.978 Scratch RF: 0.967 Sklearn RF: 0.978 **Effects**: Shallower trees, reduces noise sensitivity, faster training, minimal accuracy change.	Scratch DT: 0.936 Sklearn DT: 0.942 Scratch RF: 0.953 Sklearn RF: 0.965 **Effects**: Better generalization on noisy medical data, prevents tiny splits.	Scratch DT: 0.926 Sklearn DT: 0.944 Scratch RF: 0.944 Sklearn RF: 0.963 **Effects**: Improved stability, small accuracy gain.
RF: max_features = 0.5 (half of features)	Scratch RF: 0.956 Sklearn RF: 0.978 **Effects**: Increases tree diversity, slight accuracy drop on small dataset but reduces correlation between trees.	Scratch RF: 0.953 Sklearn RF: 0.965 **Effects**: Better handling of high-dimensional data, reduces overfitting risk.	Scratch RF: 0.944 Sklearn RF: 0.963 **Effects**: Good balance, promotes feature variety.
Handling Missing Values (Mean imputation + 10% artificial NaNs)	Scratch DT: 0.956 Sklearn DT: 0.978 Scratch RF: 0.956 Sklearn RF: 0.978 **Effects**: Minor drop vs no missing, but models remain functional; sklearn handles natively better.	Scratch DT: 0.901 Sklearn DT: 0.918 Scratch RF: 0.941 Sklearn RF: 0.953 **Effects**: Critical for real-world data; without handling, accuracy drops ~5-8%. RF more robust.	Scratch DT: 0.889 Sklearn DT: 0.907 Scratch RF: 0.926 Sklearn RF: 0.944 **Effects**: Moderate impact; imputation preserves most performance.

Observation:

The from-scratch implementations closely match sklearn's accuracy but are slower due to lack of optimizations. Tweaking parameters like max_depth prevents overfitting, while RF generally outperforms single DT by reducing variance. Entropy may be better for multi-class, Gini for binary. Handling missing values is essential for robustness. Overall, sklearn is recommended for production due to efficiency.